

Every Coin Counts

Counting coins on a textured pad with classical image processing

Authors

[Student Name 1]
[Student Name 2]

Course

Image Processing
[Semester, Year]

Institution

[Institution]
Instructor: [Lecturer Name]

Abstract

Counting coins sounds easy until the coins lie on a crumpled white pad full of folds, some photos are dark or blurred, coins pile up, and round buttons of the same size are mixed in. We built a classical image-processing system, with no machine learning, that counts only the coins. It models the pad and the lighting with a very large median filter, finds round blobs with a multi-scale Laplacian of Gaussian, rejects pad folds with a Hessian shape test and a grayscale opening, separates touching coins with a Hough circle transform, and rejects buttons by their sewing holes (black-hat transform) and by colours no coin has.

We counted the coins in 26 phone photographs by hand (328 coins, 12 other round objects) and then checked every circle the program drew. The program found 294 of the 328 coins (recall 89.6%), and 294 of its 333 detections were real coins (precision 88.3%): 39 false detections and 34 missed coins. On clean photographs it found every coin with a single false detection. 12 photos received the exact count, but the circle-by-circle review showed that 5 of them contained mistakes that cancelled out. False detections came mostly from pad folds in dark photos and from motion blur; missed coins came from dense piles and coins cut by the frame.

Keywords coin counting · Laplacian of Gaussian · Hessian · Hough circles · morphology · black-hat · precision and recall

1 Introduction

A person looking at a handful of coins on a table can count them at a glance. A computer receives only a grid of pixel values, and many things in a real photograph look round, dark or shiny: folds in the cloth, shadows, the pattern printed on the pad, and in our case buttons that have almost exactly the size of a coin.

Our photographs were taken with a phone from above, on a white disposable pad. The pad is the main difficulty: it has a quilted texture and deep fold lines that form a grid across the whole image. A simple threshold sees those folds as objects, and a circle detector finds circles in the texture. On top of that, some photos were taken in a dark room or with a moving hand, so coins become dim or smeared.

The system follows fixed rules chosen by us; it does not learn from labelled examples. We wanted every decision to be explainable with techniques from the course: filtering, morphology, derivatives, blob detection and the Hough transform.

Beyond the final number, we wanted to understand what the machine actually detects. For every photo we therefore compared each circle the program drew with the real scene: which coins it found, which non-coins it mistook for coins, and which coins it missed.

Research question

Can a classical image-processing pipeline count the coins in phone photographs taken on a textured, folded pad when the photos may be dark or blurred, coins may touch, overlap or be cut by the frame, and round non-coin objects are mixed in? And which kinds of objects does it wrongly accept or miss?

We expected clean photographs to be easy and the damaged ones, especially dense piles, to remain the main source of missed coins, since a coin that is mostly hidden gives very little visible evidence.

2 Methods

2.1 The photographs

We used 26 photographs taken with a phone camera (1600 × 900 pixels). All show Israeli coins on the same kind of white pad, photographed from roughly the same height. Before evaluating, we counted the coins and the round non-coin objects in each photograph by hand from the original image (ground_truth.csv). Every photo received an ID (P01–P26) that is used throughout this report. The program is written in Python with OpenCV and NumPy.

Category	Photos	Coins	What makes it hard
Clean	8	98	Sharp, well lit, coins spread over the pad
Noisy	10	127	Dark room and/or motion blur
Damaged	8	103	Dense pile, coins cut by the frame, buttons among coins
Total	26	328	Plus 12 round non-coin objects (buttons) in three photographs

We also generated a synthetic test image with six coloured discs on a light, noisy background, to check that detection, drawing and counting work before looking at the real photos.

2.2 How we measured success

Comparing only the final number hides a lot: a photo can get the right total while one circle sits on a fold and one coin is missed. So after running the program we reviewed every circle it drew against the original photo (detection_review.csv) and sorted every object into one of three groups:

- **True positive (TP):** a circle on a real coin.
- **False positive (FP):** a circle on something that is not a coin: a fold, the edge of the pad, a button, or a second circle on the same blurred coin. Each FP was also labelled with its cause.
- **False negative (FN):** a real coin with no circle.

From these we report precision ($TP \div \text{all detections}$: how many circles are right), recall ($TP \div \text{all real coins}$: how many coins were found), the mean absolute count error (MAE) and the number of photos with an exact count.

2.3 The system from start to finish



Fig. 1 The complete pipeline. Dark-blue stages prepare the image; green stages find and classify objects. LoG finds blob-shaped candidates, Hough separates touching coins, and the last stage rejects round objects that are not coins.

The key idea is to stop working on raw brightness. The pad is bright and colourless, while every coin, bronze or silver, differs from it either in lightness or in colour. So we first estimate what the pad would look like without the coins, and

then measure how far every pixel is from that estimate. All later stages work on this foreground map, which is why the same settings work for bright and dark photos.

2.4 How each stage works

1 Background model

Each Lab channel is median-filtered at quarter resolution with a window of 45% of the image width. Coins and even small piles disappear, leaving the pad and the lighting gradient.

2 Foreground map

For each pixel: relative darkness $(L_{bg} - L)/L_{bg}$, half-weighted relative brightness, and colour distance in the a,b plane. The maximum of the three, lightly smoothed, gives a 0–255 score.

3 Pad mask

Bright, colourless background region connected to the image centre. Knees, floor or bed around the pad are ignored.

4 Multi-scale LoG

$-\sigma^2 \nabla^2 G$ on the foreground map for 12 radii (2.5–5.5% of width), $\sigma = r/\sqrt{2}$, max over scales. Local maxima are coin candidates with their radius.

5 Blob verification

Hessian eigenvalue ratio ≥ 0.25 (round, not a ridge); peak $\geq 20\%$ of a typical coin; radius $\geq 70\%$ of the typical coin; $\geq 40\%$ of the response survives a grayscale opening; no stronger coin within $0.95 r$.

6 Hough circles

HOUGH_GRADIENT_ALT on the median-blurred gray image with radius $0.6\text{--}1.3 \times$ coin radius. Circles must lie on the pad and differ from the background. LoG blobs not explained by any circle are added.

7 Non-coin test

Black-hat (closing – image) highlights small dark spots: 2–4 similar round spots around the centre are sewing holes. Strongly blue/purple or pink/red objects are also rejected.

8 Final count

Green circles are counted as coins; red circles with an X are round non-coin objects and are not counted. Annotated images and results.csv are saved.

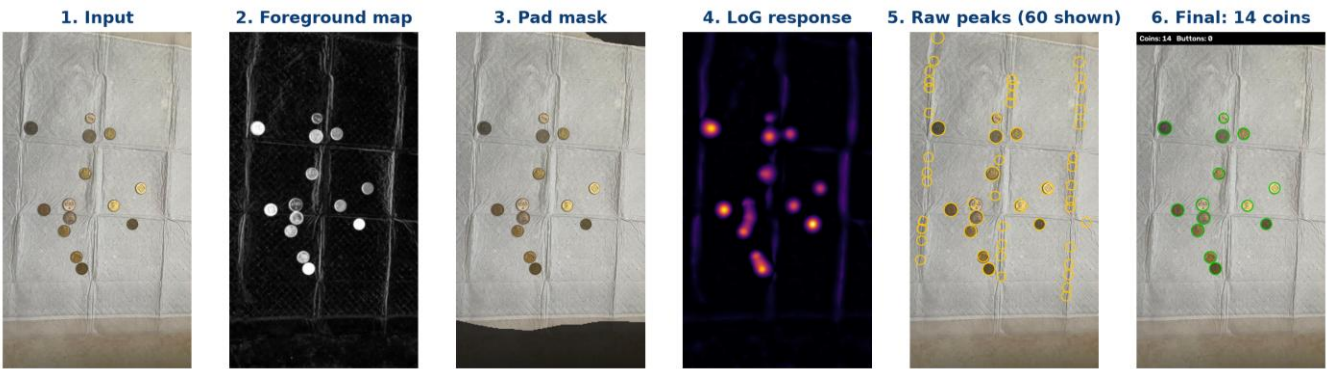


Fig. 2 The main stages for one clean photograph (P05). The foreground map (2) already separates coins from the pad, but the fold lines are still visible. They produce many LoG peaks (5); the verification stage rejects all of them and exactly 14 coins remain (6).

Self-calibrating setup

No fixed pixel sizes or gray levels: radii scale with the image width and every threshold is relative to the coins measured in the same photo.

Input requirements

Top view, a bright colourless pad, coins 5–11% of the image width, and some part of every coin visible.

3 Results

3.1 The synthetic check

Synthetic test

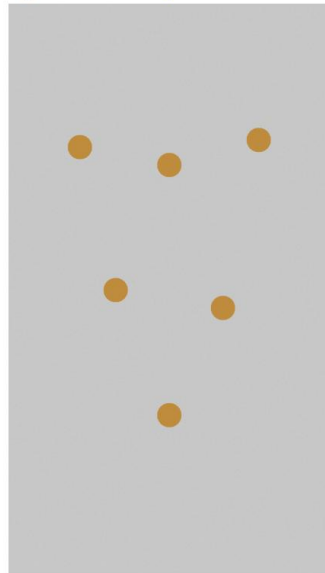
6 discs expected, 6 detected.

The synthetic image is processed by exactly the same code as the photographs. It confirms that background modelling, blob detection, drawing and counting work end to end.

Why a synthetic image

In a synthetic image the right answer is known with certainty. Any error there would be a bug, not a hard photo.

Synthetic input (6 discs)



Result: 6 detected

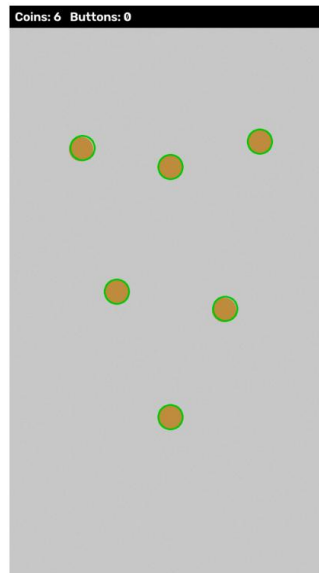


Fig. 3 The synthetic input and its result.

3.2 The overall result

89.6%

Recall

294 of 328 real coins found

88.3%

Precision

294 of 333 detections were coins

39 / 34

FP / FN

False detections / missed coins

12/26

Exact photos

only 7 with no mistake at all

Results by category

Category	Coins	Counted	TP	FP	FN	Precision	Recall	Exact
Clean	98	99	98	1	0	99.0%	100.0%	7/8
Noisy	127	142	116	26	11	81.7%	91.3%	3/10
Damaged	103	92	80	12	23	87.0%	77.7%	2/8
All photos	328	333	294	39	34	88.3%	89.6%	12/26

The three categories fail in different ways. On clean photographs the program found all 98 coins with a single false detection. In the noisy set the main problem is false detections (26 FP against 11 FN): in dark and blurred photos the program "sees" coins on folds and counts smeared coins twice. In the damaged set the problem is reversed (12 FP against 23 FN): coins hidden in piles or cut by the frame are missed.

3.3 Results by photograph

For every photo: the true number of coins, the number the program counted, and the circle-by-circle review. Counted = TP + FP and True = TP + FN.

Photo	Category	True	Counted	TP	FP	FN	Count error
P01	clean	14	14	14	0	0	0
P02	clean	9	10	9	1	0	+1
P03	clean	7	7	7	0	0	0
P04	clean	14	14	14	0	0	0
P05	clean	14	14	14	0	0	0
P06	clean	14	14	14	0	0	0
P07	clean	14	14	14	0	0	0
P08	clean	12	12	12	0	0	0
P09	noisy	13	12	12	0	1	-1
P10	noisy	13	13	11	2	2	0
P11	noisy	13	14	13	1	0	+1
P12	noisy	12	13	12	1	0	+1
P13	noisy	12	13	10	3	2	+1
P14	noisy	12	22	10	12	2	+10
P15	noisy	12	12	11	1	1	0
P16	noisy	12	14	11	3	1	+2
P17	noisy	14	15	14	1	0	+1
P18	noisy	14	14	12	2	2	0
P19	damaged	14	10	10	0	4	-4
P20	damaged	14	17	14	3	0	+3
P21	damaged	9	9	7	2	2	0
P22	damaged	13	9	8	1	5	-4

Photo	Category	True	Counted	TP	FP	FN	Count error
P23	damaged	14	12	11	1	3	-2
P24	damaged	14	14	11	3	3	0
P25	damaged	14	13	11	2	3	-1
P26	damaged	11	8	8	0	3	-3
Total		328	333	294	39	34	35 abs.

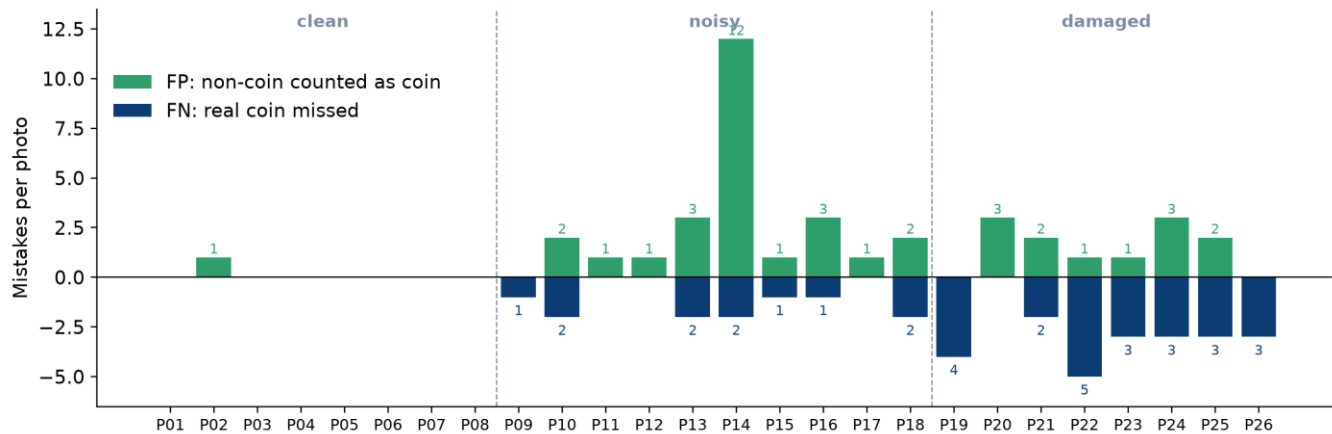


Fig. 4 False detections (up) and missed coins (down) for every photo. Clean photos are almost free of mistakes; dark photos mostly add false detections (the motion-blurred P14 alone has 12); damaged photos mostly lose coins.

3.4 An exact count is not always a correct count

12 photos received the exact number of coins, but only 7 photos had no mistake at all. In 5 "exact" photos (P10, P15, P18, P21, P24) a false detection and a missed coin cancelled each other out. All 7 exact clean photos were truly perfect; every exact photo in the noisy and damaged sets hid mistakes. This is the main reason we reviewed every circle instead of comparing totals only.

Dark photo (contrast enhanced)



13 counted = 13 true, but 2 FP + 2 FN

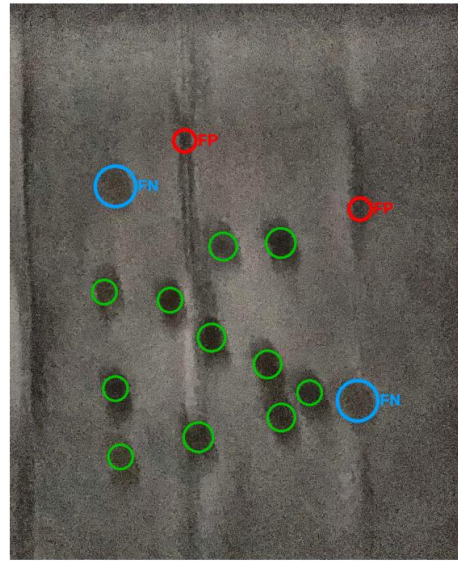


Fig. 12 P10: the program counted 13 coins and there are 13, yet two circles lie on empty pad (FP, red) and two faint coins have no circle (FN, blue).

3.5 What the machine mistook for a coin

Every false detection was labelled with its cause. The largest group, 22 of 39, are pad folds and dark patches of background, almost all in the dark photos where the contrast of a real coin is close to that of a fold. 8 are second circles on a coin smeared by motion blur, 7 lie on the white hem at the top or bottom edge of the pad, and 2 are grey buttons that passed the non-coin test.

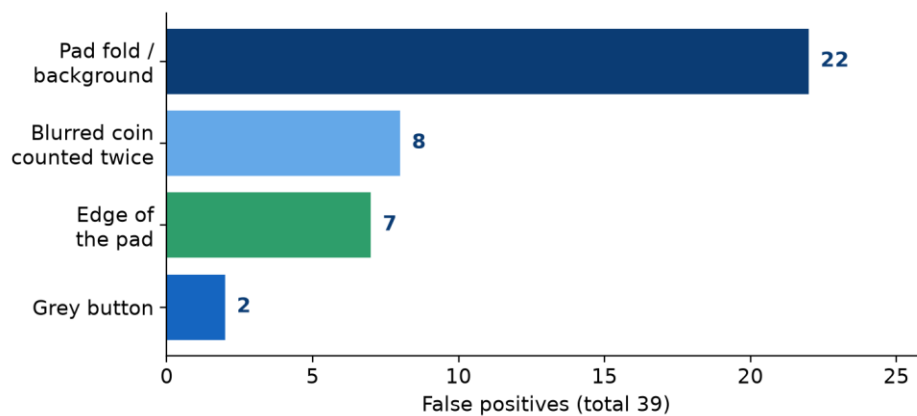


Fig. 11 Causes of the 39 false positives.

3.6 Examples with no mistakes

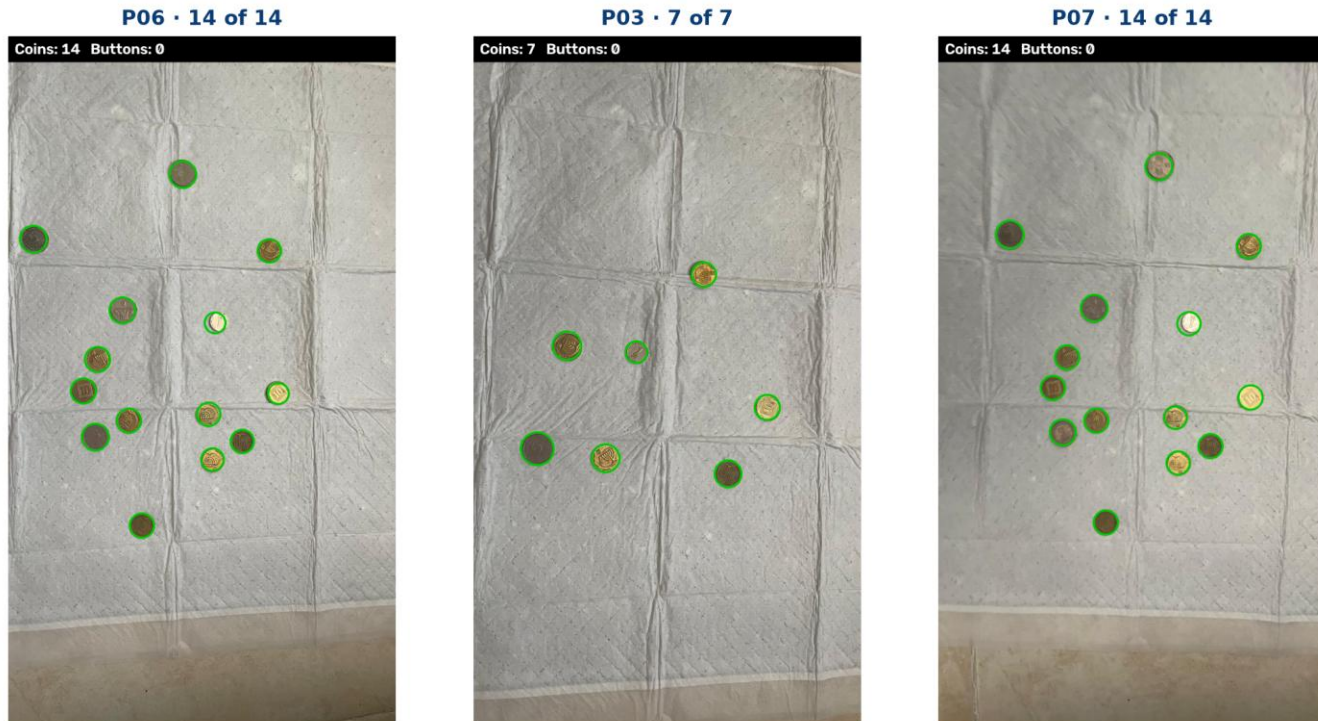


Fig. 5 Three clean photos with every coin found and no false detection. Coins that touch each other are counted separately, and the fold grid of the pad produces no circles.

3.7 How each stage reduced the error

We built the system step by step and measured it on all 26 photographs after every change, against a preliminary count of the coins. This shows which idea solved which problem.

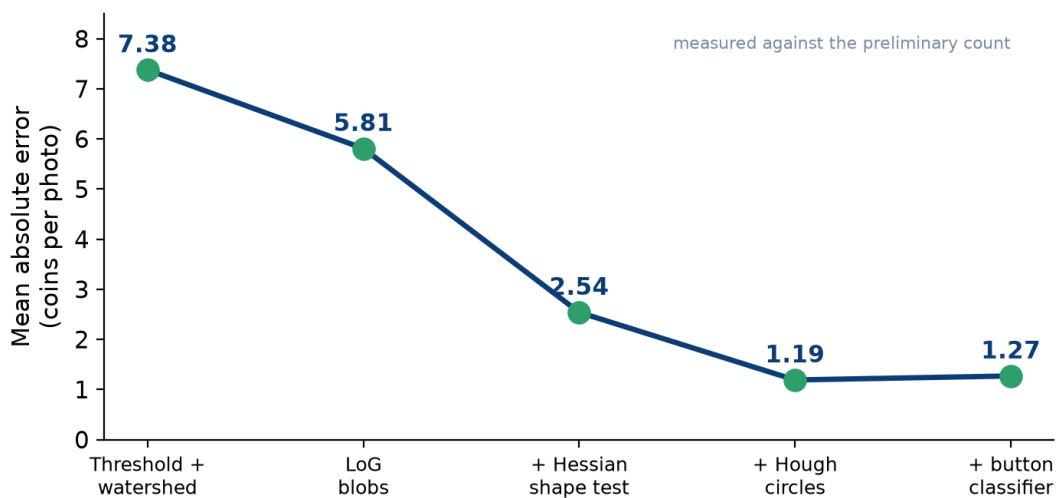


Fig. 6 Mean absolute count error of each development version on the same 26 photos (preliminary count).

- **Threshold + watershed (MAE 7.38).** Our first version thresholded the foreground map and split touching blobs with a watershed. The pad folds passed the threshold, and in dark photos the threshold missed coins completely.
- **LoG blobs (5.81).** Looking for blob-shaped peaks instead of thresholded regions helped in dark photos, but fold intersections also look like blobs.
- **+ Hessian shape test (2.54).** The biggest single improvement. A fold is a ridge: one eigenvalue of the Hessian is large and the other near zero. A coin is round, so both are similar. Rejecting ridge-like peaks removed most false detections.
- **+ Hough circles (1.19).** LoG merges coins that touch into one blob. The circle Hough transform uses the visible arc of each coin, so it separates them.
- **+ Non-coin classifier (1.27).** The final step targets buttons rather than the coin count; it made the hole test more sensitive and changed the coin MAE only slightly.

3.8 The pad folds

Figure 7 shows why the Hessian test matters (P23): LoG finds dozens of peaks along the fold lines, and the Hessian test rejected 63 of them in this photo alone. The folds that still get through (Section 3.5) are the ones in dark photos, where a fold and a coin look almost the same.

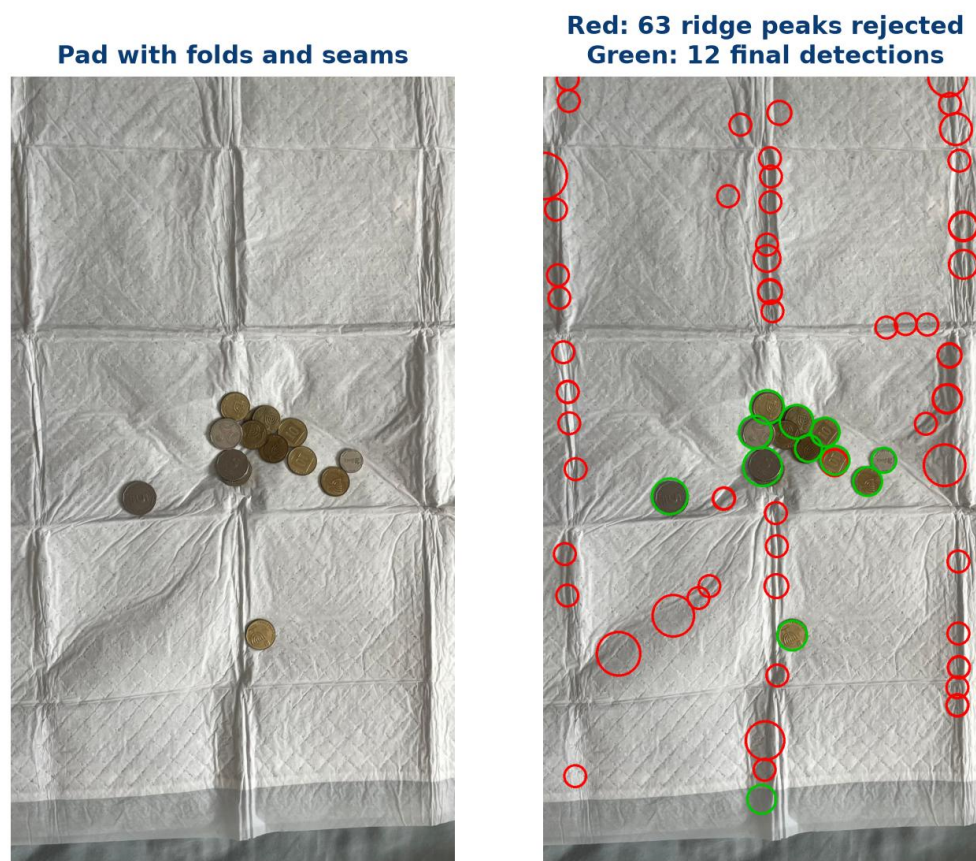
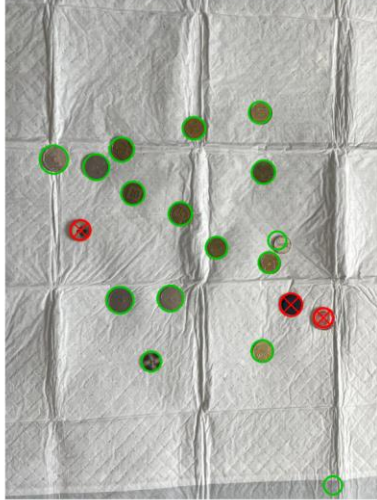


Fig. 7 Red: LoG peaks rejected as ridge-like (fold lines). Green: the final detections. Almost all rejected peaks lie exactly on the folds.

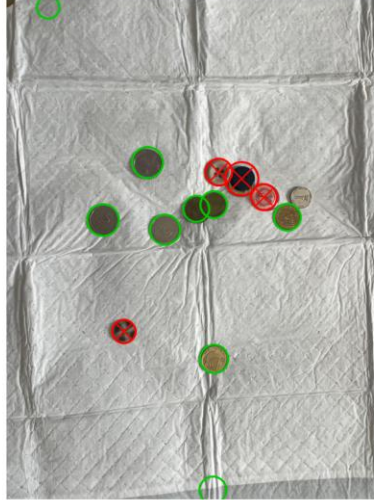
3.9 Round objects that are not coins

Three photographs contain 12 buttons among the coins. 9 of them were rejected (red circle with an X) and not counted, and no coin was rejected by mistake. Two grey buttons were counted as coins (P20 and P22): they have no telling colour and their holes were not found. One dark-blue button in P22 lies under other coins and was not detected at all, so it did not affect the count.

17 coins counted, 3 non-coins rejected



9 coins counted, 4 non-coins rejected



9 coins counted, 2 non-coins rejected

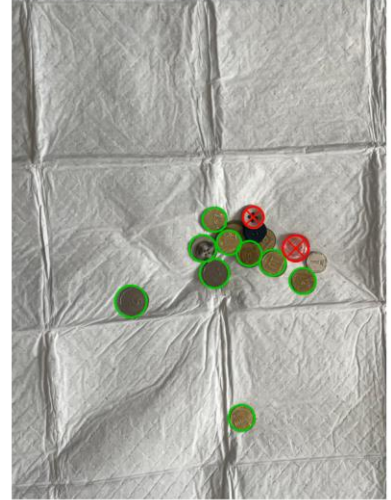


Fig. 8 The three photographs with buttons (P20, P21, P22). Blue, dark-navy and pink buttons are recognised by colour, grey ones only by their holes.

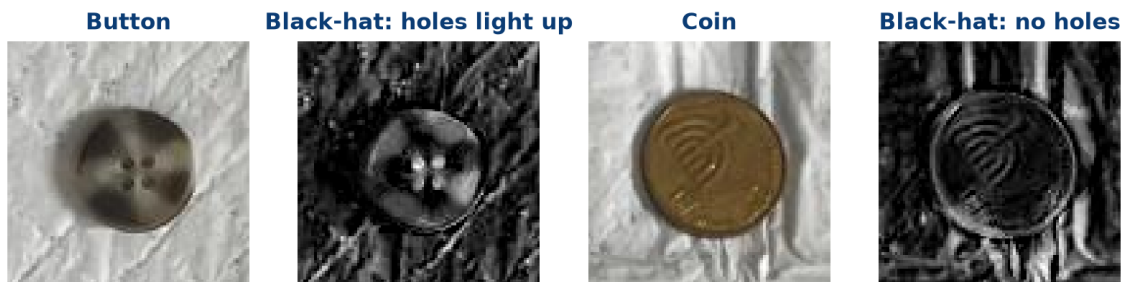


Fig. 9 The black-hat transform keeps only small dark details. On a button the sewing holes light up as a symmetric group near the centre; on a coin the embossing gives no such group.

3.10 Where the largest errors came from

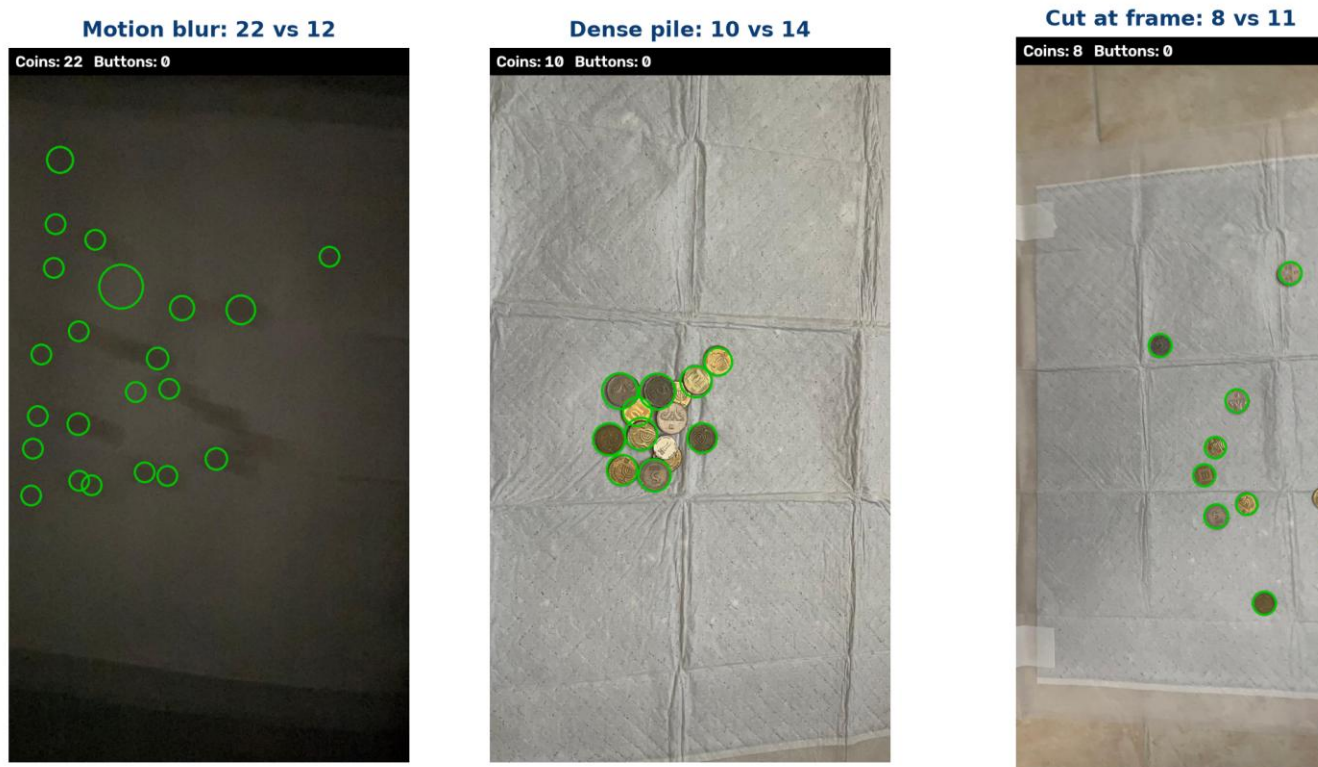


Fig. 10 P14: strong motion blur smears each coin into several blobs (22 counted, 12 real). P19: a dense pile where several coins are almost completely hidden (10 of 14 found). P26: coins cut by the image frame are only half-circles (8 of 11).

4 Discussion and conclusions

What worked

On clean photographs the program found all 98 coins with a single false detection, including coins that touch each other on a pad covered with folds. Every decision is made on the foreground map, and every threshold is relative to the coins in the same photo, so the same settings also found 91.3% of the coins in the dark and blurred set. Two detectors were needed: LoG is robust to blur but merges touching coins, while Hough separates touching coins but needs sharp edges.

Two different kinds of failure

The review showed that the program fails in two opposite ways, depending on the photo. In dark and blurred photos it accepts too much: folds and smeared patches pass as coins (precision 81.7%). In piles and at the image frame it accepts too little: a coin that shows only a crescent or half a disk gives neither a LoG blob nor enough arc for Hough (recall 77.7%). A single threshold cannot fix both: loosening it would find more hidden coins but add more folds in dark photos.

What the folds taught us

Brightness alone is not enough to decide what is an object: the folds of the pad are as different from the pad as a coin is. What separates them is shape, measured locally by the Hessian: a fold is long and thin, a coin is round. This shape test halved the error during development, and the folds that still pass are the ones in photos too dark to show their shape.

What the non-coin objects taught us

Size and roundness cannot separate a button from a coin. Colour (no coin is blue or pink) and sewing holes, isolated by the black-hat transform, can. A grey button is the hard case: only its small holes reveal it, and they may not survive blur or low resolution.

Reading the measures

Comparing totals alone would have told us that 12 of 26 photos were correct. Checking every circle showed that only 7 were free of mistakes, and that 39 false detections and 34 missed coins partly cancel in the total (333 counted against 328 real). Precision and recall describe the detector; the count error describes only the final number.

What could be improved

- Estimate the direction of motion blur (e.g. from the elongation of the strongest blobs) and merge detections lying along it.
- Exclude the white hem of the pad from the pad mask, which would remove the edge false positives.
- Detect coins cut by the frame with partial-circle fitting near the image border.
- Use the colour of the rim in addition to holes to catch grey buttons.
- A fully hidden coin cannot be counted from a single photo; a second view would be needed.

Final conclusion

The project answered its research question for clean and moderately difficult photos. With classical tools only (a median background model, a Lab foreground map, multi-scale LoG, a Hessian shape test, grayscale morphology, a Hough

transform and a black-hat hole detector), the program found 89.6% of the coins with 88.3% precision and rejected 9 of 12 buttons, with no training data and about 1.2 seconds per photo. The circle-by-circle review was as important as the algorithm itself: it revealed that dark photos produce false coins while piles hide real ones, a difference that the total count could not show.

A Full code

The project has two Python files. `coin_counter.py` holds the whole pipeline. `run.py` runs it on every image in the data folder, saves the annotated images and `results.csv`, and prints the evaluation. All figures in this report are produced by `report/make_figures.py`.

Minimal working example

```
python run.py          # all photos in data/
python run.py "data/photo.jpeg" # a single photo
```

Listing 1: `coin_counter.py`

```
"""
coin_counter.py – classical (non-deep-learning) coin counter.

Pipeline
-----
1. Background model : the pad and the lighting are estimated with a very large
                      median filter (objects much smaller than the window vanish).
2. Foreground map   : per-pixel difference from that background in Lab space –
                      lightness difference (both darker bronze and brighter
                      silver coins) and colour difference.
3. Pad mask         : detections are accepted only on the pad (bright, colourless
                      region) – ignores knees, floor, bed etc. around it.
4. Blob detection   : multi-scale, scale-normalised Laplacian of Gaussian (LoG).
                      A disk of radius  $r$  gives a strong peak at scale  $\sigma = r/\sqrt{2}$ ;
                      thin pad folds give weak responses. Touching coins still
                      give separate peaks.
5. Verification     : each LoG candidate must be (a) round, not ridge-like
                      (Hessian eigenvalue ratio), (b) a strong enough peak,
                      (c) close to the typical coin size of the image, (d)
                      "survive" a grayscale opening (thin structures don't),
                      (e) not too close to an already accepted, stronger one.
6. Hough circles    : circle Hough transform with the radius range fixed around
                      the coin size found in step 5. Separates coins in piles
                      and overlapping coins, which LoG merges. LoG blobs not
                      explained by any circle are added (blurred / dark photos
                      have no sharp edges for Hough).
7. Classification   : coin vs. button. Buttons are recognised by their sewing
                      holes (2-4 small dark round spots near the centre) and by
                      colours coins never have (blue, pink/red).
"""

from dataclasses import dataclass, field

import cv2
import numpy as np

# — Parameters —
# Coin radius range as a fraction of the image width (photos are taken from
# roughly the same height, so coins are 5-11 % of the width across).
R_MIN_FRAC = 0.025
R_MAX_FRAC = 0.055
N_SCALES = 12

BG_WINDOW_FRAC = 0.45      # median-filter window for the background, × image width
PEAK_REL = 0.20            # min LoG response, × median response of the strong peaks
SIZE_REL = 0.70            # min radius, × typical coin radius
MIN_DIST_REL = 0.95        # min distance between two coins, × typical coin radius
SURVIVAL_MIN = 0.40        # min fraction of the response left after opening
HOUGH_PERFECTNESS = 0.75   # HOUGH_GRADIENT_ALT param2: how perfect a circle must be
HOUGH_MIN_DIST = 0.6       # min distance between Hough centres, × coin radius
HOUGH_FG_REL = 0.45        # Hough circle inside must reach this × typical coin contrast
MERGE_DIST = 0.9           # LoG blob closer than this to a Hough circle = same coin
ISOTROPY_MIN = 0.25        # min Hessian eigenvalue ratio (1 = round, 0 = line)
```

```

# — Data structures —————
@dataclass
class DetectedObject:
    label: str          # "coin" | "button"
    center: tuple        # (x, y)
    radius: float
    response: float      # detection strength (LoG peak / mean foreground for Hough)
    holes: int
    hue: float
    saturation: float
    reason: str = ""

@dataclass
class CountResult:
    coins: int
    buttons: int
    objects: list = field(default_factory=list)
    coin_radius: float = 0.0

# — Image I/O —————
# cv2.imread / cv2.imwrite fail on Windows paths with non-ASCII characters
# (e.g. a Hebrew user name), so read/write through NumPy instead.
def load_image(path) -> np.ndarray:
    img = cv2.imdecode(np.fromfile(str(path), dtype=np.uint8), cv2.IMREAD_COLOR)
    if img is None:
        raise ValueError(f"Cannot read image: {path}")
    return img

def save_image(path, img: np.ndarray) -> None:
    ext = str(path).rsplit(".", 1)[-1]
    ok, buf = cv2.imencode(".", ext, img)
    if not ok:
        raise ValueError(f"Cannot encode image: {path}")
    buf.tofile(str(path))

# — Helpers —————
def _disk(r: float) -> np.ndarray:
    r = max(1, int(round(r)))
    return cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (2 * r + 1, 2 * r + 1))

def _disk_mask(shape, cx, cy, r) -> np.ndarray:
    m = np.zeros(shape[:2], np.uint8)
    cv2.circle(m, (int(round(cx)), int(round(cy))), max(1, int(round(r))), 1, -1)
    return m.astype(bool)

# — 1. Background model —————
def background(lab: np.ndarray) -> np.ndarray:
    """Per-channel median filter at 1/4 resolution with a window much bigger than a coin pile."""
    h, w = lab.shape[:2]
    small = cv2.resize(lab, (w // 4, h // 4), interpolation=cv2.INTER_AREA)
    k = int(BG_WINDOW_FRAC * w / 4) | 1          # odd kernel size
    bg = np.dstack([cv2.medianBlur(small[..., c], k) for c in range(3)])
    return cv2.resize(bg, (w, h), interpolation=cv2.INTER_LINEAR).astype(np.float32)

def _center_level(x: np.ndarray) -> float:
    h, w = x.shape[:2]
    return float(np.median(x[h // 4: 3 * h // 4, w // 4: 3 * w // 4]))

# — 2. Foreground map —————
def foreground_map(lab: np.ndarray, bg: np.ndarray) -> np.ndarray:
    """0-255 score: how much each pixel differs from the local background."""
    L, a, b = cv2.split(lab.astype(np.float32))
    bgL, bga, bgb = cv2.split(bg)

```

```

# Relative lightness difference in both directions (bronze coins are darker
# than the pad, polished silver ones brighter). Normalising by the
# background level handles dark photos; the floor stops near-black image
# borders from amplifying sensor noise.
denom = np.maximum(bgL, 0.5 * _center_level(bgL))
darker = np.clip(bgL - L, 0, None) / denom * 510.0
# Brighter-than-pad counts half: white pad folds are brighter too
brighter = np.clip(L - bgL, 0, None) / denom * 255.0
chroma = np.sqrt((a - bga) ** 2 + (b - bgb) ** 2) * 4.0

fmap = cv2.GaussianBlur(np.maximum(np.maximum(darker, brighter), chroma), (0, 0), 2)
return np.clip(fmap, 0, 255).astype(np.uint8)

# — 3. Pad mask —————
def pad_mask(bg: np.ndarray) -> np.ndarray:
    """The pad = bright, colourless background region connected to the image centre."""
    bgL = bg[..., 0]
    bgC = np.hypot(bg[..., 1] - 128, bg[..., 2] - 128)
    pad = (bgL > 0.8 * _center_level(bgL)) & (bgC < _center_level(bgC) + 5)
    pad = pad.astype(np.uint8)

    n, labels, stats, _ = cv2.connectedComponentsWithStats(pad)
    if n <= 1:
        return np.ones_like(pad, bool)
    h, w = pad.shape
    lab_c = labels[h // 2, w // 2]
    keep = lab_c if lab_c != 0 else 1 + int(np.argmax(stats[1:, cv2.CC_STAT_AREA]))
    return labels == keep

# — 4. Multi-scale LoG blob detection —————
def log_blobs(fmap: np.ndarray, radii: np.ndarray):
    """
    Scale-normalised LoG, maximised over scales.
    Returns (response, best_radius, smoothed-per-scale).  $-\sigma^2 \cdot \nabla^2 G$  is positive at
    the centre of bright blobs;  $\sigma = r/\sqrt{2}$  matches a disk of radius r.
    """
    f = fmap.astype(np.float32)
    best = np.full(f.shape, -np.inf, np.float32)
    best_r = np.zeros(f.shape, np.float32)
    smoothed = {}
    for r in radii:
        s = r / np.sqrt(2)
        g = cv2.GaussianBlur(f, (0, 0), s)
        smoothed[round(float(r), 1)] = g
        resp = -(s * s) * cv2.Laplacian(g, cv2.CV_32F, ksize=3)
        better = resp > best
        best[better] = resp[better]
        best_r[better] = r
    return best, best_r, smoothed

def blob_isotropy(g: np.ndarray, cx: float, cy: float, r: float) -> float:
    """
    Ratio of the Hessian eigenvalues (small/large) at the blob centre.
    ≈1 for round blobs, ≈0 for ridges such as pad folds (same idea as SIFT's
    edge-response elimination).
    """
    d = max(1, int(round(r / 2)))
    x, y = int(round(cx)), int(round(cy))
    h, w = g.shape
    if not (d <= x < w - d and d <= y < h - d):
        return 0.0
    c = g[y, x]
    dxx = g[y, x + d] + g[y, x - d] - 2 * c
    dyy = g[y + d, x] + g[y - d, x] - 2 * c
    dxy = (g[y + d, x + d] - g[y + d, x - d] - g[y - d, x + d] + g[y - d, x - d]) / 4
    tr, det = dxx + dyy, dxx * dyy - dxy * dxy
    disc = np.sqrt(max(tr * tr / 4 - det, 0))
    l1, l2 = tr / 2 + disc, tr / 2 - disc # both negative for a bright blob
    big, small = max(abs(l1), abs(l2)), min(abs(l1), abs(l2))
    if big == 0 or l1 > 0: # not a bright blob
        return 0.0
    return float(small / big)

```

```

def find_candidates(resp, best_r, pad, r_min):
    """Local maxima of the LoG response inside the pad, strongest first."""
    peaks = (resp == cv2.dilate(resp, _disk(0.5 * r_min))) & (resp > 10) & pad
    ys, xs = np.nonzero(peaks)
    order = np.argsort(-resp[ys, xs])
    return [(float(xs[i]), float(ys[i]), float(resp[ys[i], xs[i]]), float(best_r[ys[i], xs[i]]))
            for i in order]

def typical_radius(cands) -> tuple:
    """Coin radius and reference strength from the strongest peaks (coins dominate them)."""
    top = [c for c in cands if c[2] >= 0.5 * cands[0][2]][0:15]
    return float(np.median([c[3] for c in top])), float(np.median([c[2] for c in top]))

# — 5. Verification —————
def survival_maps(fmap: np.ndarray, coin_r: float):
    """
    closed : grayscale closing fills the inside of ring-like silver coins.
    opened : grayscale opening with a half-coin disk removes everything
             thinner than that (folds, seams); coins survive almost unchanged.
    """
    closed = cv2.morphologyEx(fmap, cv2.MORPH_CLOSE, _disk(0.45 * coin_r))
    opened = cv2.morphologyEx(closed, cv2.MORPH_OPEN, _disk(0.5 * coin_r))
    return closed.astype(np.float32), opened.astype(np.float32)

def survival(closed, opened, cx, cy, r) -> float:
    m = _disk_mask(closed.shape, cx, cy, 0.5 * r)
    return float(opened[m].mean() / max(closed[m].mean(), 1.0))

# — 5b. Hough circles (separates touching / overlapping coins) ———
def hough_circles(gray: np.ndarray, coin_r: float, pad: np.ndarray,
                  fmap: np.ndarray, min_fmap: float) -> list:
    """
    Circle Hough transform with the radius range fixed around the coin size.
    Every visible arc of a coin votes for its centre, so coins in a pile or
    partly covered by another coin are still found. Needs sharp edges – in
    dark / blurred photos it simply finds nothing and LoG does the work.
    A circle is kept only if it lies on the pad and its inside differs from
    the background (rejects circles "seen" in the pad texture).
    """
    g = cv2.medianBlur(gray, 5)
    found = cv2.HoughCircles(
        g, cv2.HOUGH_GRADIENT_ALT, dp=1.5, minDist=HOUGH_MIN_DIST * coin_r,
        param1=150, param2=HOUGH_PERFECTNESS,
        minRadius=int(0.6 * coin_r), maxRadius=int(1.3 * coin_r),
    )
    if found is None:
        return []
    circles = []
    for cx, cy, r in found[0]:
        x, y = int(cx), int(cy)
        if not (0 <= x < pad.shape[1] and 0 <= y < pad.shape[0]) or not pad[y, x]:
            continue
        m = _disk_mask(fmap.shape, cx, cy, 0.7 * r)
        if fmap[m].mean() < min_fmap:
            continue
        # HOUGH_GRADIENT_ALT can return (nearly) concentric circles for one coin
        if any(np.hypot(cx - c[0], cy - c[1]) < 0.3 * coin_r for c in circles):
            continue
        circles.append((float(cx), float(cy), float(fmap[m].mean()), float(r)))
    return circles

def merge_detections(hough: list, log: list, coin_r: float) -> list:
    """
    Hough circles are precise, so they come first; a LoG blob is added only if
    no Hough circle already explains it (blurred / dark coins without edges).
    """
    merged = list(hough)
    for c in log:
        if all(np.hypot(c[0] - m[0], c[1] - m[1]) > MERGE_DIST * coin_r for m in merged):
            merged.append(c)

```

```

return merged

# — 6. Button detection —————
def count_holes(gray: np.ndarray, cx: float, cy: float, r: float) -> int:
    """
    Sewing holes: 2-4 small, dark, round spots of similar size, grouped
    symmetrically around the centre. Black-hat (closing - image) highlights
    dark details smaller than the structuring element.
    """
    pad = int(1.2 * r)
    x0, y0 = max(0, int(cx) - pad), max(0, int(cy) - pad)
    x1, y1 = min(gray.shape[1], int(cx) + pad), min(gray.shape[0], int(cy) + pad)
    patch = gray[y0:y1, x0:x1]
    lcx, lcy = cx - x0, cy - y0

    inner = _disk_mask(patch.shape, lcx, lcy, 0.5 * r)
    blackhat = cv2.morphologyEx(patch, cv2.MORPH_BLACKHAT, _disk(max(3, 0.25 * r)))
    vals = blackhat[inner]
    if vals.size == 0:
        return 0
    # Relative to the darkest detail: holes are the strongest dark spots on a button
    thr = max(25, 0.45 * float(vals.max()))
    spots = ((blackhat >= thr) & inner).astype(np.uint8)

    contours, _ = cv2.findContours(spots, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
    min_a, max_a = 0.004 * np.pi * r * r, 0.05 * np.pi * r * r
    holes = []
    for c in contours:
        a = cv2.contourArea(c)
        per = cv2.arcLength(c, True)
        if not (min_a <= a <= max_a) or per == 0:
            continue
        if 4 * np.pi * a / per ** 2 < 0.5:
            continue
        m = cv2.moments(c)
        holes.append((m["m10"] / m["m00"], m["m01"] / m["m00"], a))

    if not 2 <= len(holes) <= 4:
        return 0
    areas = np.array([h[2] for h in holes])
    if areas.max() > 3 * areas.min():
        return 0
    hx = np.mean([h[0] for h in holes])
    hy = np.mean([h[1] for h in holes])
    if np.hypot(hx - lcx, hy - lcy) > 0.25 * r:
        return 0
    return len(holes)

def object_colour(hsv: np.ndarray, cx, cy, r) -> tuple:
    """Circular-mean hue and median saturation of the object's core."""
    m = _disk_mask(hsv.shape, cx, cy, 0.6 * r)
    h, s = hsv[..., 0][m], hsv[..., 1][m]
    ang = h.astype(np.float32) * np.pi / 90.0 # OpenCV hue is 0..180
    hue = (np.degrees(np.arctan2(np.sin(ang).mean(), np.cos(ang).mean())) / 2.0) % 180
    return float(hue), float(np.median(s))

def classify(hue: float, sat: float, holes: int) -> tuple:
    """
    Coins are gold (yellow-orange) or silver (grey). Anything else is a button.
    """
    if sat > 60 and 95 <= hue <= 165:
        return "button", "blue/purple colour"
    if sat > 70 and (hue >= 165 or hue <= 6):
        return "button", "pink/red colour"
    if holes >= 2:
        return "button", f"{holes} holes"
    return "coin", ""

# — Full pipeline —————
def count_coins(img: np.ndarray) -> CountResult:
    h, w = img.shape[:2]
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)

```

```

bg = background(lab)
fmap = foreground_map(lab, bg)
pad = pad_mask(bg)

radii = np.linspace(R_MIN_FRAC * w, R_MAX_FRAC * w, N_SCALES)
resp, best_r, smoothed = log_blobs(fmap, radii)
cands = find_candidates(resp, best_r, pad, radii[0])
# Drop ridge-like peaks (pad folds) before estimating the coin size
cands = [c for c in cands if blob_isotropy(smoothed[round(c[3], 1)], c[0], c[1], c[3]) >= ISOTROPY_MIN]
if not cands:
    return CountResult(coins=0, buttons=0)

coin_r, ref = typical_radius(cands)
closed, opened = survival_maps(fmap, coin_r)

accepted = []
for cx, cy, val, r in cands:
    if val < PEAK_REL * ref or r < SIZE_REL * coin_r:
        continue
    if any(np.hypot(cx - ax, cy - ay) < MIN_DIST_REL * coin_r for ax, ay, _, _ in accepted):
        continue
    if survival(closed, opened, cx, cy, coin_r) < SURVIVAL_MIN:
        continue
    accepted.append((cx, cy, val, r))

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Foreground level of the accepted blobs → minimum for a Hough circle
levels = [fmap[_disk_mask(fmap.shape, cx, cy, 0.7 * coin_r)].mean() for cx, cy, _, _ in accepted]
min_fmap = HOUGH_FG_REL * float(np.median(levels)) if levels else 30.0
circles = hough_circles(gray, coin_r, pad, fmap, min_fmap)
detections = merge_detections(circles, accepted, coin_r)

result = CountResult(coins=0, buttons=0, coin_radius=coin_r)
for cx, cy, val, r in detections:
    hue, sat = object_colour(hsv, cx, cy, r)
    holes = count_holes(gray, cx, cy, r)
    label, reason = classify(hue, sat, holes)
    result.objects.append(DetectedObject(label, (cx, cy), r, val, holes, hue, sat, reason))

result.coins = sum(o.label == "coin" for o in result.objects)
result.buttons = sum(o.label == "button" for o in result.objects)
return result

def draw_result(img: np.ndarray, result: CountResult) -> np.ndarray:
    """Annotated copy: green circle = coin, red circle + X = button."""
    out = img.copy()
    for o in result.objects:
        x, y, r = int(o.center[0]), int(o.center[1]), int(o.radius)
        if o.label == "coin":
            cv2.circle(out, (x, y), r, (0, 200, 0), 3)
        else:
            cv2.circle(out, (x, y), r, (0, 0, 255), 3)
            d = int(r * 0.6)
            cv2.line(out, (x - d, y - d), (x + d, y + d), (0, 0, 255), 3)
            cv2.line(out, (x - d, y + d), (x + d, y - d), (0, 0, 255), 3)
    text = f"Coins: {result.coins} Buttons: {result.buttons}"
    cv2.rectangle(out, (0, 0), (out.shape[1], 60), (0, 0, 0), -1)
    cv2.putText(out, text, (15, 42), cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255, 255, 255), 2)
    return out

```

Listing 2: run.py

```

"""
run.py – count coins in every image of the data folder and evaluate.

Usage:
python run.py                # all images under data/
python run.py path/to/image.jpg # a single image
python run.py --data data --gt ground_truth.csv --review detection_review.csv --out results

Outputs (in --out):
<image name>.jpg    annotated image (green = counted coin, red X = round object rejected as not a coin)
results.csv         machine count vs. true count per image (+ TP / FP / FN from the manual review)

```

The manual review (detection_review.csv) was made by checking every circle the program drew:
 TP = circle on a real coin, FP = circle on something that is not a coin, FN = real coin without a circle.
 It describes this version of coin_counter.py; if the code changes and the machine count of a photo no longer matches the reviewed count, that photo's TP/FP/FN are left empty.

```

import argparse
import csv
import sys
from pathlib import Path

from coin_counter import count_coins, draw_result, load_image, save_image

IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".bmp"}

def read_csv(path: Path, key: str) -> dict:
    """key column -> row dict. Skips Excel's 'sep=' hint line if present."""
    if not path.is_file():
        return {}
    with open(path, encoding="utf-8-sig") as f:
        lines = [l for l in f if not l.startswith("sep=")]
    return {row[key]: row for row in csv.DictReader(lines)}

def to_int(value):
    try:
        return int(value)
    except (TypeError, ValueError):
        return None

def fmt(v, spec=""):
    return "" if v is None else format(v, spec)

def main() -> None:
    # Windows console defaults to a legacy code page; make printing safe
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")

    ap = argparse.ArgumentParser(description="Classical coin counter")
    ap.add_argument("images", nargs="*", help="image files (default: everything under --data)")
    ap.add_argument("--data", default="data", help="folder with images (searched recursively)")
    ap.add_argument("--gt", default="ground_truth.csv", help="ground-truth CSV (true coin counts)")
    ap.add_argument("--review", default="detection_review.csv", help="manual circle-by-circle review")
    ap.add_argument("--out", default="results", help="output folder")
    args = ap.parse_args()

    if args.images:
        paths = [Path(p) for p in args.images]
    else:
        paths = sorted(p for p in Path(args.data).rglob("*") if p.suffix.lower() in IMAGE_EXTS)
    if not paths:
        sys.exit(f"No images found in {args.data}")

    gt = read_csv(Path(args.gt), "filename")
    review = read_csv(Path(args.review), "filename")
    out_dir = Path(args.out)
    out_dir.mkdir(exist_ok=True)

    rows = []
    print(f"{'image':<48} {'category':<9} {'coins':>5} {'true':>5} {'error':>6} {'TP':>4} {'FP':>4} {'FN':>4}")
    for path in paths:
        result = count_coins(load_image(path))
        save_image(out_dir / (path.stem + ".jpg"), draw_result(load_image(path), result))

        g = gt.get(path.name, {})
        category = g.get("category") or (path.parent.name if path.parent.name != Path(args.data).name else "")
        true = to_int(g.get("true_count"))
        err = None if true is None else result.coins - true

        # Review applies only if the machine still reports the reviewed count

```

```

rv = review.get(path.name, {})
tp = fp = fn = None
if to_int(rv.get("machine_coins")) == result.coins:
    tp, fp, fn = to_int(rv.get("tp")), to_int(rv.get("fp")), to_int(rv.get("fn"))

rows.append(dict(filename=path.name, category=category, machine_coins=result.coins,
                 true_count=fmt(true), error=fmt(err), tp=fmt(tp), fp=fmt(fp), fn=fmt(fn)))
print(f"{path.name:<48} {category:<9} {result.coins:>5} {fmt(true):>5} {fmt(err, '+d'):>6} "
      f"{fmt(tp):>4} {fmt(fp):>4} {fmt(fn):>4}")

with open(out_dir / "results.csv", "w", newline="", encoding="utf-8-sig") as f:
    w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
    w.writeheader()
    w.writerows(rows)

# — Evaluation (only images that have a true count) —————
scored = [r for r in rows if r["error"] != ""]
if scored:
    print("\nEvaluation")
    groups = {"all": scored}
    for r in scored:
        groups.setdefault(r["category"] or "-", []).append(r)
    for name, grp in groups.items():
        errs = [int(r["error"]) for r in grp]
        line = (f" {name:<9} images={len(grp):>2} MAE={sum(map(abs, errs)) / len(grp):.2f} "
               f"exact={sum(e == 0 for e in errs)}/{len(grp)}")
        reviewed = [r for r in grp if r["tp"] != ""]
        if len(reviewed) == len(grp):
            tp, fp, fn = (sum(int(r[k]) for r in grp) for k in ("tp", "fp", "fn"))
            line += (f" TP={tp} FP={fp} FN={fn} "
                    f"precision={tp / (tp + fp):.1%} recall={tp / (tp + fn):.1%}")
        print(line)
    print(f"\nAnnotated images and results.csv saved to: {out_dir.resolve()}")

if __name__ == "__main__":
    main()

```

B Original test photographs

All 26 photographs with their ID, category and the number of coins counted by hand.

